



Distributed probabilistic top- k dominating queries over uncertain databases

Niranjan Rai¹ · Xiang Lian¹ 

Received: 10 May 2021 / Revised: 22 May 2023 / Accepted: 11 June 2023 /

Published online: 1 July 2023

© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2023

Abstract

In many real-world applications such as business planning and sensor data monitoring, one important, yet challenging, task is to rank objects (e.g., products, documents, or spatial objects) based on their ranking scores and efficiently return those objects with the highest scores. In practice, due to the unreliability of data sources, many real-world objects often contain noises and are thus imprecise and uncertain. In this paper, we study the problem of *probabilistic top- k dominating* (PTD) query on such large-scale uncertain data in a *distributed* environment, which retrieves k uncertain objects from *distributed uncertain databases* (on multiple distributed servers), having the largest ranking scores with high confidences. In order to efficiently tackle the distributed PTD problem, we propose a MapReduce framework for processing distributed PTD queries over distributed uncertain databases. In this MapReduce framework, we design effective pruning strategies to filter out false alarms in the distributed setting, propose cost-model-based index distribution mechanisms over servers, and develop efficient distributed PTD query processing algorithms. Extensive experiments have demonstrated the efficiency and effectiveness of our proposed distributed PTD approaches on both real and synthetic data sets through various experimental settings.

Keywords Distributed probabilistic top- k dominating query · Distributed uncertain database

1 Introduction

Nowadays, large volumes of imprecise and uncertain data have been produced in many real-world applications such as sensor networks, location-based services (LBS), moving object tracking, information extraction, and so on. The uncertainty in these application data is not uncommon, due to various reasons such as noises introduced during the data collection/transmission [5], inconsistent/conflicting public opinions/comments [2, 14], or the data perturbation/generalization [23] intentionally manipulated by humans for privacy-preserving

✉ Xiang Lian
xlian@kent.edu

Niranjan Rai
nrai@kent.edu

¹ Kent State University, Kent, USA

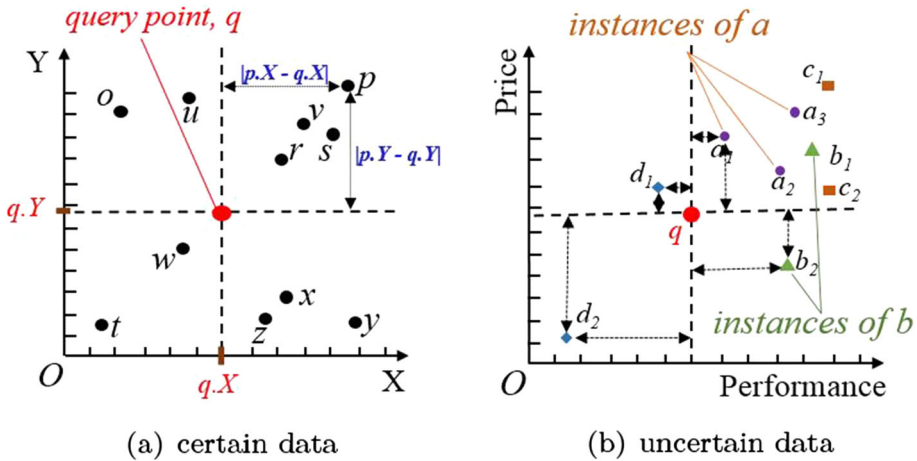


Fig. 1 An example of top- k dominating queries over certain and uncertain data

purposes. Therefore, it is very important, yet challenging, to study how to manage and query such big uncertain data efficiently and effectively.

In this paper, we consider a useful and important spatial query, *top- k dominating query* [27, 43], over a large-scale uncertain database in a distributed environment, which has many real-life applications such as multi-criteria decision making [38], coal mine surveillance [27], and so on.

Figure 1a shows an example of a top- k dominating query over 2-dimensional (certain) data points $\{o, p, r, s, t, u, v, w, x, y, z\}$. Given a query point q , each object (e.g., p) has 2 *dynamic attributes* w.r.t. q , given by $p.A_1 = |p.X - q.X|$ and $p.A_2 = |p.Y - q.Y|$. The top- k dominating query retrieves k objects with the highest ranking scores, where the score of each object is defined by the number of objects that are worse than this object, in terms of dynamic attributes (i.e., the number of objects dynamically dominated by this object, as will be later discussed in Definition 2). Intuitively, the query point q is the preferred target specified by a user, and all objects are ranked based on the user's preferences w.r.t. q (i.e., the ranking scores based on dynamic attributes).

Due to the pervasive uncertainty in real-world data, in this paper, we will consider *probabilistic top- k dominating query* (PTD) [27] over large-scale and distributed uncertain database [6], which obtains k uncertain objects with the highest scores that are defined under the *possible worlds* semantics [10]. The PTD query has many real applications such as decision making in business.

We have the following motivation example, shown as follows.

Example 1 (Business Planning for the Laptop Market) Figure 1b shows an example of customers' preferences to laptops in a 2D performance-and-price space. As plotted in the figure, each customer group (e.g., a corporation or a school), say a , has a number of customers with different laptop preferences (e.g., a_1, a_2 , and a_3), which can be obtained from user surveys. Table 1 depicts an uncertain database that contains customer preferences about laptops in Fig. 1b, where customer groups correspond to uncertain objects, and laptop preferences by customers in each group can be treated as instances of the uncertain object (customer group). Each customer preference is associated with its price and performance attributes, as well as an appearance probability which indicates the portion of customers in the group who prefer

Table 1 The uncertain database in Fig. 1b

Uncertain object t (customer group)	Instance t_j (performance, price)	Appearance probability $t_i \cdot p$
a	a_1 (7, 10)	0.3
	a_2 (9, 8)	0.3
	a_3 (10, 11)	0.4
b	b_1 (8, 11)	0.5
	b_2 (11, 9)	0.5
c	c_1 (11, 12)	0.4
	c_2 (11, 7)	0.6
d	d_1 (5, 8)	0.2
	d_2 (2, 2)	0.8

this performance-and-price setting. As an example, instance a_1 from the customer group (uncertain object) a has performance-and-price attributes (7, 10), which are preferred by 30% of customers in the group a . Due to the large scale of customer group data in different spatial locations (e.g., store in different cities), we consider the storage and analysis of such laptop preference data of customer groups in a distributed environment (e.g., cloud).

Assume that a laptop company wants to produce a new laptop model, q , and would like to find k customer groups who are more likely to buy this model q . Note that, in real applications of online advertising and marketing, under the constraint of budget limit, the company may target a limited number of k customer groups (who are most interested in the new product q), and push advertisement or coupons to them to increase the sales volume. As shown in Fig. 1b, given a new laptop model (query point) q (with targeted performance and price), the company can conduct a PTD query over the uncertain database in Table 1 to obtain k customer groups that are expected to have preferences closer to the new model q than most of the other groups.

There are many other applications for the PTD query, such as ranking NO_2 and SO_2 concentrations in the air [34], and the coal mine surveillance application [27].

Inspired by examples above, in this paper, we will study the PTD problem under a distributed setting. In particular, due to the large scale of uncertain databases or inherent features that data are collected and stored at different servers, we assume that the uncertain database is distributed on multiple servers. The distributed PTD problem retrieves k uncertain objects with the highest (expected) ranking scores from distributed uncertain database (stored on different servers), using the MapReduce framework. Compared with centralized PTD query processing over a single server, PTD on the distributed uncertain database can achieve lower response time, by taking advantage of parallel computations among multiple servers. Also, in this paper, we consider the dynamic top- k dominating query, where users can input a desired query point which may be more beneficial to the user compared to the static top- k dominating query where users cannot give an input query point and all the results are returned compared to the origin (0,0).

The distributed PTD problem is challenging, due to the large scale, uncertainty, and the distributing environment of the underlying data. One straightforward method is to simply collect all the uncertain objects from distributed servers, compute scores of all uncertain objects, and finally return k uncertain objects with the highest ranking scores. However,

this straightforward method incurs high communication cost which consumes much network bandwidth (due to the heavy workload of the data transfer among servers). Moreover, the score calculation of each object involves all other objects, it is also rather costly (i.e., with quadratic cost) to compute exact scores for all uncertain objects, especially in large-scale distributed uncertain databases.

In order to efficiently and effectively tackle the PTD problem over the distributed uncertain database, in this paper, we will divide data into different partitions, create an index over each partition, and propose a novel cost-model-based index distribution technique to reduce the search space of the PTD problem. We will also devise an efficient algorithm under a MapReduce framework that can effectively query on distributed uncertain databases and return the top- k dominating uncertain objects.

In summary, we make the following contributions in this paper.

- We formally define the problem of probabilistic top- k dominating query (PTD) over distributed uncertain database (rather than the centralized uncertain database) in Sect. 2.
- We devise effective pruning methods for distributed PTD processing in Sect. 3, and propose a MapReduce framework for answering PTD queries in the distributed uncertain database in Sect. 4.
- We design an effective indexing mechanism for distributed PTD query processing and provide a cost model for guiding index distribution in Sect. 5.
- We propose efficient algorithms in Sect. 6 for the filtering-mapper, filtering-reducer, and refinement-mapper phases which follow the MapReduce framework.
- We evaluate the performance of our PTD approaches through extensive experiments in Sect. 7.

Section 8 reviews the related works on top- k dominating queries on centralized certain/uncertain data and distributed certain data, as well as queries on distributed uncertain data. Section 9 concludes this paper.

2 Problem definition

In this section, we define *probabilistic top- k dominating query* (PTD) on the distributed uncertain database.

2.1 Distributed uncertain database

An uncertain database consists of uncertain objects, each of which contains a number of *mutually exclusive* instances associated with existence probabilities. Due to the large scale of the uncertain database, we assume that uncertain objects reside in N servers/nodes at different physical locations, have the same schema, and are connected by a network, which is called a distributed uncertain database.

Definition 1 (Distributed Uncertain Database) A distributed uncertain database D is a collection of N uncertain databases $D_1, D_2, \dots, D_N$, which contain the same schema and are located in N servers in a network, respectively. Each uncertain database D_l ($1 \leq l \leq N$) contains M_l uncertain objects. Each uncertain object $t \in D_l$ can be represented by an instance set $\{t_1, t_2, \dots, t_{|t|}\}$, and each instance t_i ($1 \leq i \leq |t|$) in t contains d numerical attributes $t_i.A_1, t_i.A_2, \dots, t_i.A_d$, as well as its appearance probability $t_i.p$, satisfying $\sum_{i=1}^{|t|} t_i.p \leq 1$.

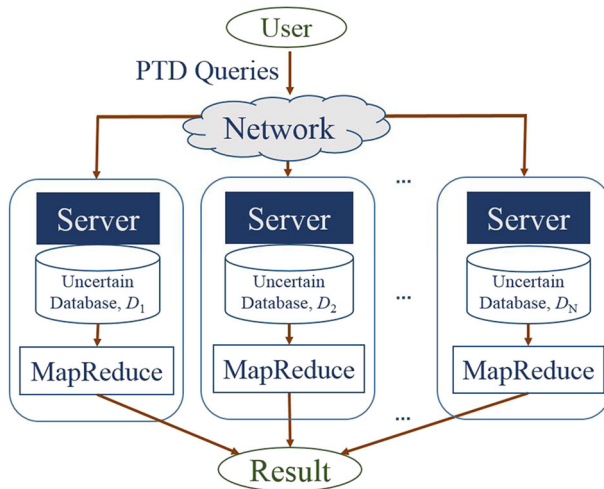


Fig. 2 The PTD framework in the distributed uncertain database

Example 2 Figure 2 depicts an example of a distributed network that connects N uncertain databases D_i ($1 \leq i \leq N$) located at N server nodes, respectively. All these uncertain databases collectively form a distributed uncertain database D . Each uncertain database D_i contains uncertain objects, for example, a , b , c , and d , as shown in Table 1.

2.2 Dynamic dominance

In this subsection, we define the dynamic dominance relationship between two instances in the distributed uncertain database.

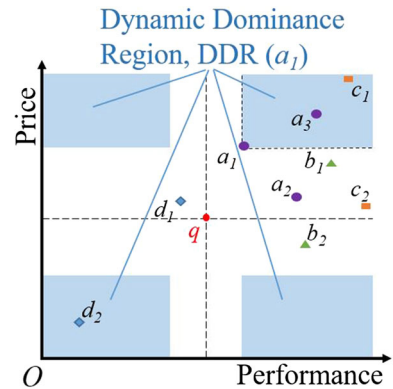
Definition 2 (*Dynamic Dominance Between 2 Instances*) Given two instances u_i and v_j from uncertain objects u and v , respectively, and a query point q , we say that u_i *dynamically dominates* v_j with respect to q , denoted as $u_i \prec_q v_j$, if two conditions hold: (1) for every dimension k ($1 \leq k \leq d$), we have $|u_i.A_k - q.A_k| \leq |v_j.A_k - q.A_k|$ and (2) there exists at least one dimension k ($1 \leq k \leq d$) such that $|u_i.A_k - q.A_k| < |v_j.A_k - q.A_k|$ holds.

Intuitively, in Definition 2, the query point q is a preferred target of the user (e.g., targeted hotel or expected new product), based on which the dynamic attributes of other instances are calculated. We thus define the dynamic attributes of an instance, for example, u_i , as the absolute coordinate differences between u_i and query point q (i.e., $|u_i.A_k - q.A_k|$). If all dynamic attributes of instance u_i are not worse than that of an instance v_j , and u_i has at least one dynamic attribute strictly better than v_j , then we say that instance u_i dynamically dominates instance v_j with respect to q .

Example 3 In Fig. 3, the query point q is the preference object used for defining dynamic attributes of instances. Instance a_1 dynamically dominates instance c_1 , since $|a_1.A_k - q.A_k| < |c_1.A_k - q.A_k|$ holds for both dimensions $k = 1, 2$.

Equivalently, in Example 3, with respect to q , we can draw symmetric points a'_1 , a''_1 , and a'''_1 of instance a_1 . Then, we can define the *dynamic dominance regions*, $\text{DDR}(a_1)$, with respect to a_1 , a'_1 , a''_1 , and a'''_1 , which are shown as the shaded regions in Fig. 3. This way, any instance (e.g., c_1) falling in $\text{DDR}(a_1)$ is dynamically dominated by instance a_1 .

Fig. 3 An example of dynamic dominance



2.3 Ranking scores of instances/objects

In this subsection, we first provide the score definition for an instance of an uncertain object, and then formally define the ranking score of an uncertain object in an uncertain database. Since each uncertain object contains one or multiple instances with appearance probabilities, we need to take into account the probabilistic information when defining the score of an instance or an uncertain object.

Definition 3 (The Score of an Instance) For a given instance t_j of an uncertain object t , its score, $S(t_j)$, is given by the appearance probability $t_j.p$ of t_j times the summation of appearance probabilities for instances that are dynamically dominated by instance t_j :

$$\begin{aligned}
 S(t_j) &= \sum_{\forall s_i \in D \wedge s \neq t} \sum_{\forall pw(D) \wedge t_j \in pw(D) \wedge s_i \in pw(D)} Pr\{pw(D)\} \cdot \chi(t_j \prec_q s_i), \\
 &= t_j.p \cdot \sum_{\forall s_i \in D \wedge s \neq t} Pr\{t_j \prec_q s_i\} \\
 &= t_j.p \cdot \sum_{\forall s_i \in D \wedge s \neq t} s_i.p \cdot \chi(t_j \prec_q s_i)
 \end{aligned} \tag{1}$$

where $\chi(z) = 1$, if z is true; $\chi(z) = 0$, otherwise.

Under the possible worlds semantics [13], the score $S(t_j)$ of an instance t_j is given by the summation of appearance probabilities for possible worlds $pw(D)$ such that t_j dynamically dominates other instances s_i in $pw(D)$. We can further rewrite this formula on the instance level, by multiplying the appearance probability $t_j.p$ with the probability, $Pr\{\cdot\}$, that t_j dynamically dominates other instances s_i , where $Pr\{\cdot\}$ is given by the summation of probabilities $s_i.p \cdot \chi(t_j \prec_q s_i)$. Intuitively, higher appearance probability, $t_j.p$, of an instance t_j or more instances dominated by t_j leads to a higher score $S(t_j)$ of instance t_j .

Example 4 In Fig. 3, since instance a_1 dynamically dominates c_1 and d_2 w.r.t. q , by Definition 3, the ranking score, $S(a_1)$, of instance a_1 is $a_1.p \cdot (d_2.p + c_1.p) = 0.3 \times (0.8 + 0.4) = 0.36$.

After defining the score for the instance, we next give the score of the uncertain object by summing up the scores of its instances.

Definition 4 (Ranking Score of an Uncertain Object) The ranking score $S(t)$ of an uncertain object t is given by the summation of scores of all the instances t_j in object t , that is,

$$S(t) = \sum_{j=1}^{|t|} S(t_j). \quad (2)$$

In Example 4, the score of instance a_1 is given by $S(a_1) = 0.36$. Similarly, we have: $S(a_2) = 0.66$ and $S(a_3) = 0.48$. Thus, the score of uncertain object a is given by $S(a) = S(a_1) + S(a_2) + S(a_3) = 0.36 + 0.66 + 0.48 = 1.5$.

2.4 Probabilistic top- k dominating query in the distributed environment

Next, we define the *probabilistic top- k dominating query* (PTD) [27] over a large-scale uncertain database in the distributed environment. Assume that we have N servers where each server stores a subset, D_i (for $1 \leq i \leq N$), of the distributed uncertain database D .

Definition 5 (PTD over the Distributed Uncertain Database) Given a distributed uncertain database D on N server nodes, that is, $D = \{D_1, D_2, \dots, D_N\}$, and an integer k , a *probabilistic top- k dominating* (PTD) query retrieves k uncertain objects, t , from the distributed uncertain database D with the highest ranking scores $S(t)$, where $S(t)$ is given by Definition 4.

One straightforward method to tackle the PTD problem over the distributed uncertain database (as given by Definition 5) is to collect all uncertain objects from N server nodes D_i and return top- k dominating uncertain objects over the distributed uncertain database D . However, in doing so, we must face several challenges. First, since uncertain data are distributed over different servers in the network, it is challenging to reduce the communication cost of the PTD query answering over the network. Second, due to the data uncertainty, efficient PTD query processing over distributed uncertain databases is also challenging, due to the exponential number of *possible worlds* [10]. In other words, it is not trivial how to efficiently retrieve the PTD answers over data associated with probabilities. Third, it is also challenging to design an effective and efficient distributed algorithm (e.g., following the MapReduce framework) to answer PTD queries over uncertain data.

3 Pruning methods for distributed PTD processing

In this section, we first provide the pruning heuristics for our distributed PTD query processing algorithms. Since the distributed uncertain database D is located at N different servers, in order to enable efficient PTD processing, we aim to filter out as many PTD false alarms as possible on each server locally, which results in only a few PTD candidates to be refined among N servers (with minimal communication cost). These pruning methods can be also used for the filtering of false alarms over distributed servers.

Distributed PTD pruning Specifically, PTD over distributed uncertain database D retrieves k uncertain objects $t \in D$ with the highest scores $S(t)$ (as given in Eq. (4)). Thus, our basic idea is to retrieve at least k PTD answer candidates from each partition D_i , and other objects in D_i that are not candidates can be safely pruned.

Lemma 1 (Distributed PTD Pruning) *Given distributed uncertain database $D = \{D_1, D_2, \dots, D_N\}$, if we conduct the PTD query over each partition D_l on local server and obtain a set, A_l , of k PTD answers (with the highest scores) only over D_l , then any uncertain object $t \in D_l$ that is not in A_l can be safely pruned.*

Proof According to the lemma assumption, A_l contains k PTD answers over each local partition D_l . Then, the actual PTD answers (k uncertain objects with the highest ranking scores) must be a subset of the retrieved ($N \cdot k$) candidates from all the N partitions. Therefore, other objects $t \in D_l$, which are not in the PTD answer set A_l , cannot be the actual PTD answers over D and thus can be safely pruned. $\square$

Score bound pruning While the calculation of the score $S(t)$ for an object $t \in D_l$ may involve objects in D located on N different servers (in the worst case), it is challenging to efficiently compute the exact scores $S(t)$ across multiple servers with low communication cost. Therefore, alternatively, our basic pruning idea is to efficiently derive lower and upper bounds of the score $S(t)$, denoted as $t.LB$ and $t.UB$, respectively, and then utilize them to quickly rule out those uncertain objects o with low ranking scores $S(o)$.

We have the following lemma about the pruning method by using the score bounds.

Lemma 2 (Score Bound Pruning) *Assume that we have obtained a set, S_{cand} , of k PTD candidates $t^{(1)}, t^{(2)}, \dots$, and $t^{(k)}$ in uncertain database $D_l \subseteq D$, and τ_l is the k largest score lower bound among these candidates. Then, any uncertain object $o \in D_l$ can be safely pruned, if it holds that $o.UB < \tau_l$.*

Proof Since $o.UB$ is the upper bound of score $S(o)$, we have $S(o) \leq o.UB$. Similarly, we can obtain $t^{(i)}.LB \leq S(t^{(i)})$ for k PTD candidates in S_{cand} , where $1 \leq i \leq k$.

Moreover, since τ_l is the k largest score lower bound among k candidates $t^{(1)}, t^{(2)}, \dots$, and $t^{(k)}$, it holds that $t^{(i)}.LB \geq \tau_l$ for all objects $t^{(i)}$. By the transitivity, we can obtain $\tau_l \leq t^{(i)}.LB \leq S(t^{(i)})$ for all $1 \leq i \leq k$.

According to the lemma assumption that $o.UB < \tau_l$, by inequality transition, it holds that $S(o) \leq o.UB < \tau_l \leq S(t^{(i)})$ for $1 \leq i \leq k$. That is, uncertain object o has its ranking score $S(o)$ lower than at least k PTD candidates in S_{cand} . Thus, based on Definition 5, object o definitely cannot be in the PTD answer set and can be safely pruned. $\square$

Lemma 2 provides the pruning condition to quickly discard those uncertain objects with low ranking scores (i.e., $< \tau_l$). To enable this score bound pruning method, one important remaining issue is on how to obtain tight score lower/upper bounds (i.e., $t.LB$ and $t.UB$, respectively) of an uncertain object t in a distributed environment.

Derivation of score lower/upper bounds Since the computation of the lower/upper bounds of score $S(t)$ involves objects across multiple servers, it is not possible to directly calculate the score, or its bounds, of an uncertain object t over only one single partition D_l at a local server. Therefore, as will be discussed later in Sect. 5.1, we will build an aggregate R-tree (aR-tree) index [22], I_l , over each partition D_l and distribute it to other servers. This way, each server will store a partition D_l (and its index I_l), and meanwhile maintain summaries (i.e., aR-tree) of $(N - 1)$ other partitions on servers.

With partition D_l and N aR-tree indexes $I_1, I_2, \dots$, and I_N , we next discuss how to compute the score lower/upper bounds, $t.LB$ and $t.UB$, of an uncertain object $t \in D_l$. In particular, since the score $S(t)$ is given by summing up the scores $S(t_j)$ of instances $t_j \in t$, the score lower/upper bounds $t.LB$ and $t.UB$ can be given by summing up score lower/upper bounds

of instances, $t_j.LB$ and $t_j.UB$, respectively. That is, we have:

$$t.LB = \sum_{\forall t_j \in t} t_j.LB, \text{ and} \quad (3)$$

$$t.UB = \sum_{\forall t_j \in t} t_j.UB. \quad (4)$$

Given a query point q , partition D_l , and N indexes $I_1 \sim I_N$, the score lower/upper bounds, $t_j.LB$ and $t_j.UB$, of instance t_j can be calculated as follows. With respect to q and instance t_j , we can obtain the dynamic dominance region $DDR(t_j)$ (see an example in Fig. 3). We consider the bottom level of index I_i that is stored on the current server. For any *minimum bounding rectangle* (MBR) node e in index I_i (for $i \neq l$), if it holds that e is fully contained in $DDR(t_j)$, then the number of uncertain objects, $e.sum$, under node e will be counted in the score lower bound $t_j.LB$. Moreover, through index I_l over partition D_l (at local server), the summed probability of all instances in $DDR(t_j)$ will be added to $t_j.LB$. Formally, we have:

$$\begin{aligned} t_j.LB = & \sum_{\forall i \in [1, N] \wedge i \neq l} \sum_{\forall e \in I_i \wedge e \subseteq DDR(t_j)} e.sum \\ & + \sum_{\forall o_r \in I_l \wedge o_r \subseteq DDR(t_j)} o_j.p. \end{aligned} \quad (5)$$

Note that, in Eq. (5), we underestimate the score $S(t_j)$ by ignoring those MBRs e that are *partially dominated* by t_j . Here, given an instance t_j , an MBR e , and a query point q , we say that MBR e is partially dominated by t_j (with respect to query point q), if there exists at least one dimension k ($1 \leq k \leq d$), such that $|e.A_k^- - q.A_k| \leq |t_j.A_k - q.A_k| \leq |e.A_k^+ - q.A_k|$ holds, where $e.A_k^-$ and $e.A_k^+$ are minimum and maximum possible values of MBR e along the k -th dimension, respectively. Therefore, Eq. (5) only considers those MBRs e that are fully dominated by t_j , but not that partially dominated by t_j , which results in a score lower bound, $t_j.LB$.

Similarly, we can compute the score upper bound, $t_j.UB$, as the number, $e.sum$, of objects in those MBRs $e \in I_i$ intersecting with $DDR(t_j)$ (or the summed probability of instances that fall into $DDR(t_j)$).

$$\begin{aligned} t_j.UB = & \sum_{\forall i \in [1, N] \wedge i \neq l} \sum_{\forall e \in I_i \wedge e \cap DDR(t_j) \neq \emptyset} e.sum \\ & + \sum_{\forall o_r \in I_l \wedge o_r \subseteq DDR(t_j)} o_j.p. \end{aligned} \quad (6)$$

Intuitively, Eq. (6) computes a score upper bound $t_j.UB$, which overestimates the score, $S(t_j)$, by considering those MBRs e both fully and partially dominated by t_j with respect to query point q (i.e., intersecting with $DDR(t_j)$).

4 The distributed PTD processing framework

In this section, we present a framework for PTD query answering over distributed servers in Algorithm 1. Specifically, our framework consists of four phases: *offline pre-processing phase*, *filtering-mapper phase*, *filtering-reducer phase*, and *refinement-mapper phase*.

In the first phase, we offline divide the uncertain database D into N disjoint partitions $\{D_1, D_2, \dots, D_N\}$, based on our proposed cost model, and distribute them to N servers, respectively (lines 1–2). For each partition D_l , we will construct an aR-tree index I_l , and send the index I_l to $(N - 1)$ other servers. In order to minimize the communication cost of the distributed PTD query processing (and save the space cost as well), we will also design a cost model to select the most appropriate levels of tree indexes I_l and send to other servers (lines 3–5).

In the filtering-mapper phase, given any PTD query, for each partition D_l , we will traverse the index I_l , and meanwhile apply the score bound pruning (discussed in Sect. 3) to reduce the PTD search space and retrieve a candidate set S_{cand} of PTD answers (line 7). Since each server containing partition D_l only has indexes (summaries) of other partitions D_l (for $l \neq i$), we cannot calculate the exact scores, $S(t)$, of candidates $t \in S_{cand}$. Thus, in this phase, we will send instances t_j of candidates $t \in S_{cand}$, as well as those MBRs eid in partition D_l that are partially dominated by t_j , to the reducer server containing D_l (line 8).

Next, in the filtering-reducer phase, each server containing partition D_l will calculate the partial score of instances $t_j \in D_l$, with respect to the query point q and MBRs eid (line 10). In the refinement-mapper phase, each server will receive the partial scores of instances t_j from the same uncertain objects t , and then combine partial scores into the exact scores, $S(t)$, of uncertain objects t (lines 11–13).

Finally, k uncertain objects t with the highest scores $S(t)$ will be returned as PTD query answers (line 14).

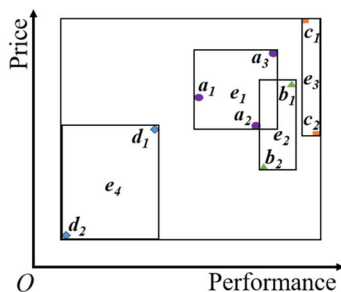
5 Offline pre-processing

In this section, we will discuss in Sect. 5.1 how to perform the indexing in the offline pre-processing phase of Algorithm 1. Then, we will provide the cost model for guiding the index distribution among servers in Sect. 5.2.

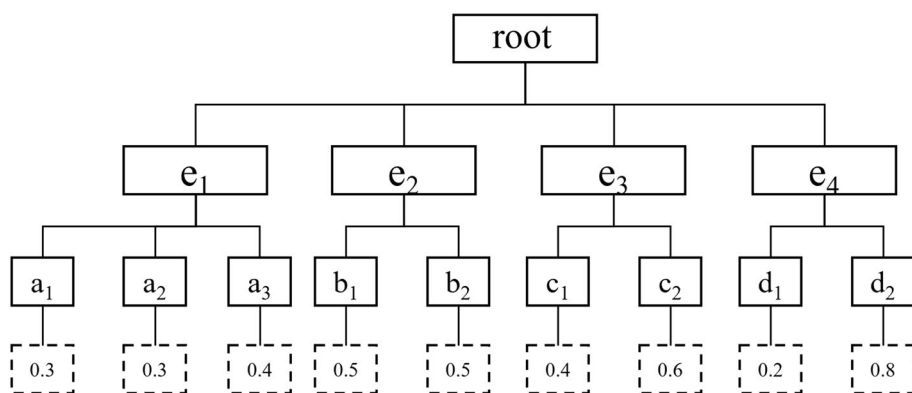
5.1 Indexing mechanism

In this paper, we will randomly assign uncertain objects from the database to N different partitions, denoted as D_l , which are then distributed to N servers, respectively, where $1 \leq l \leq N$. Note that, we tested other partitioning strategies (e.g., space partitioning or pivot-based partitioning), and found that the random partitioning can achieve the best pruning power, efficiency, and communication cost. In this paper, we consider static data in the uncertain database. Therefore, we only need to build the index once, during the offline pre-processing phase, and this index can support online PTD queries with respect to dynamic query points multiple times. The index is thus independent with the query, and if the query point q changes, we can still re-use the offline constructed index to answer queries, as shown in Fig. 4. Even in the case of dynamic data (e.g., with data insertion/deletions), we can also incrementally update our index with insertion of new objects and deletion of uncertain objects, which can be used to support the PTD queries.

To facilitate efficient PTD query processing on each server, we will build an *aggregate R-Tree* (aR-Tree) [22], I_l , over each partition D_l . Specifically, for each uncertain object $t \in D_l$, we use a *minimum bounding rectangle* (MBR) to minimally bound all its instances $t_j \in t$ (with existence probabilities $t_j.p \in [0, 1]$), and insert this MBR (associated with a COUNT aggregate: 1) into the aR-tree I_l . As a result, each leaf node e in the aR-tree index contains



(a) An example of MBRs



(b) aR-Tree

Fig. 4 The aR-Tree construction

MBRs of uncertain objects t (each associated with a COUNT aggregate 1); each non-leaf node e contains MBR entries e_r , which minimally bound all uncertain objects under e_r , and are associated with COUNT aggregates, $e_r.sum$, under entries e_r .

Figure 4a shows an example of MBRs that are built in a partition. In Fig. 4b, we can see an example of aR-Tree built from the MBRs shown in Fig. 4a. Here, each uncertain object contains instances which has their existential probability (also known as appearance probability) associated with them. Thus, the aR-Tree nodes store the (SUM) aggregates of appearance probabilities of its child nodes. For example, node e_1 stores an aggregate 1, which is the summed appearance probability of all objects under e_1 . Similarly, non-leaf node e_2 is associated with a SUM aggregate 1.

5.2 Cost-model-based index distribution

When the index I_l over D_l is constructed, we will send it to $(N - 1)$ other servers (in order to calculate score lower/upper bounds for objects in other partitions). In order to minimize the communication cost, in this subsection, we will design a cost model to select the appropriate levels of the tree index I_l , that minimizes the *communication cost*, CC , of PTD query answering, and distribute them to other servers.

Algorithm 1: The Distributed PTD Query Framework

Input: a distributed uncertain database D , the number, N , of servers, a query point q , and parameter k
Output: top- k dominating uncertain objects with highest scores

```

// Offline Pre-Processing Phase (Sect. 5)
1  $\{D_1, D_2, \dots, D_N\} = \text{Data\_Partitioning}(D, N)$ 
2 send each partition,  $D_l$  ( $1 \leq l \leq N$ ), to a server
3 for each partition  $D_l$  do
4   | construct an aR-Tree index,  $I_l$ 
5   | send appropriate levels of index  $I_l$  to  $(N - 1)$  other servers
// Filtering-Mapper Phase (Sect. 6.1)
6 for each partition  $D_i$  do
7   | apply pruning methods to retrieve a candidate set,  $S_{cand}$ , of PTD query answers from  $D_i$  via index  $I_i$ 
8   | send instances,  $t_j$ , of candidates  $t \in S_{cand}$  to other partitions  $D_l$  ( $l \neq i$ ) that contain partially dominated MBRs  $eid$ 
// Filtering-Reducer Phase (Sect. 6.2)
9 for each partition  $D_l$  do
10  | calculate the partial score of instances  $t_j \in D_i$ , w.r.t. query point  $q$  and MBRs  $eid$ 
// Refinement-Mapper Phase (Sect. 6.3)
11 for each uncertain object  $t$  without actual score do
12  | receive partial scores of instances  $t_j$  from the same uncertain objects  $t$ 
13  | compute the actual score  $S(t)$  of uncertain objects  $t$ 
14 emit  $k$  uncertain objects with the highest ranking score  $S(t)$ 

```

Specifically, for each partition D_l , we build an aggregate R-Tree (aR-Tree) index and share this tree with other servers. While sending the index tree, we must decide whether to send the finer or coarser level of this tree to different servers. If we send the finer level of the tree, we will obtain tighter bounds of the scores, which leads to higher pruning power (i.e., smaller number of candidates). However, on the other hand, finer level results in more index nodes, which have higher chances to be partially dominated by instance and in turn higher communication costs. Similarly, if we send the coarser level of the index tree, then we may have smaller number of MBRs that might be partially dominated by the instance (i.e., incurring lower communication cost), but lower pruning power (PP) (i.e., leading to higher communication cost). So, we define a cost model which estimates the CC for sharing different levels of the tree indexes and select the best levels of the index tree with minimal communication cost. Let $S_{MBR}(q)$ be a list of MBRs that cannot be dynamically dominated by other MBRs (w.r.t. query point q) in partition D_l . The Communication Cost, CC , of the distributed PTD processing can be given by:

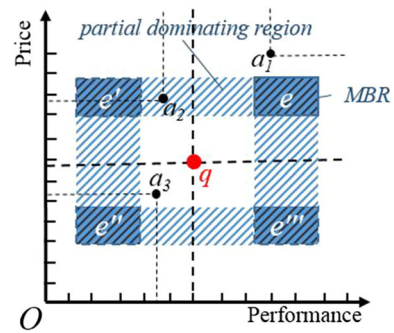
$$CC = \sum_{l=1}^N C_l, \quad (7)$$

where C_l is the number of instances, t_i , of the uncertain objects, t , in a candidate set, S_{cand} , that partially dominate region of MBR $e \in S_{MBR}$. It can be calculated as:

$$C_l = \sum_{e \in S_{MBR}(q)} N_{ins}(e, |S_{cand}|). \quad (8)$$

In Eq. (8), function $N_{ins}(e, |S_{cand}|)$ returns the number of instances in the candidate set S_{cand} , which partially dominate the MBR e . We will collect the distribution of online query points from historical query log (note: if query logs are not available, we will use the

Fig. 5 An example of the partial dominating region (PDR)



Uniform distribution of query points to estimate the C_l value), and take the average of the communication cost, C_l , over partition D_l , with respect to different possible locations of query points. C_l in Eq. (8) plays a vital role in determining the communication cost of the PTD query, because higher value of C_l leads to higher CC .

Before we discuss how to compute function $N_{ins}(e, |S_{cand}|)$, we first introduce a notion, called *partial dominating region* (PDR), with respect to a query point q and an MBR e . Figure 5 illustrates an example of the PDR in the shaded region, in which e' , e'' , and e''' are symmetric mapping MBRs w.r.t. query point q and MBR e . We say that the shaded region is a *partial dominating region* (PDR), since any instance (e.g., a_2) falling into PDR must partially dominate MBR e (and in turn incur the communication cost during PTD query processing).

To calculate $N_{ins}(e, |S_{cand}|)$ for each MBR e , we will use a one-pass algorithm [19] to find those instances that partially dominate MBR e (i.e., in the PDR), which should be output by the mapper phase. Here, we will consider the worst-case scenario, which scans all the instances in PDR, and returns $|S_{cand}|$ uncertain objects with the highest numbers of instances falling in the PDR. This way, we can compute the total number of instances among the returned $|S_{cand}|$ objects that appear in PDR, which is exactly the output of function $N_{ins}(e, |S_{cand}|)$.

Finally, the only remaining issue is how to compute the number, $|S_{cand}|$, of candidate uncertain objects in each partition D_l , after the pruning. By considering the number of pruned objects for partition D_l and applying *central limit theorem* (CLT) [42], we can obtain the number of the remaining candidates:

$$\begin{aligned} |S_{cand}| &= |D_l| \cdot (1 - \Pr\{t.UB < \tau_l \mid \forall t \in D_l\}) \\ &= |D_l| \cdot \left(1 - \Phi\left(\frac{E(\tau_l) - \mu}{\sigma^2}\right)\right). \end{aligned} \quad (9)$$

where $\Phi(\cdot)$ is the *cumulative density probability* (cdf) of standard normal distribution, $E(\tau_l)$ is the expected value of score threshold τ_l for partition D_l , and μ and σ are the mean and variance, respectively, of a random variable for score upper bounds $t.UB$ of uncertain objects t in D_l .

6 Distributed PTD algorithms

The distributed PTD algorithm consists of MapReduce jobs in two rounds: the job in the first round includes filtering-mapper and the filtering-reducer functions, and the job in the second round is a map-only mapper function, namely refinement-mapper function.

6.1 Filtering-mapper function

In this subsection, we will present the algorithm for first mapper function: filtering-mapper (as discussed in the PTD framework of Algorithm 1), which obtains at least k PTD candidates over each partition D_i on the i -th server (via indexes I_l for $1 \leq l \leq N$).

Specifically, Algorithm 2 illustrates the pseudo code of the filtering-mapper phase, which prunes false alarms of PTD answers at each uncertain database D_i , and obtains a set of at least k PTD candidates from each server for refinement. To do so, we insert the root to the heap and until the heap is empty we calculate the score upper bound of MBRs and uncertain objects to check if it contains uncertain objects and finally return the k + candidate uncertain objects and the MBRs that are only partially dominated by the uncertain objects. In particular, the input of Algorithm 2 includes the partition number i as key and uncertain database D_i , aR-Tree indexes I_l of all other partitions, a query point q , and a parameter k as the value, whereas the output of the algorithm includes a list of key-value pairs containing instances t_j of PTD candidates t and their partially dominated MBRs eid , which are also the input of the mapper phase.

In Algorithm 2, for each uncertain database D_i , we maintain an initially empty candidate set S_{cand} , the size, cnt , of the candidate set, and a pruning threshold τ_i (line 1). Moreover, we keep a (initially empty) *minimum heap* H with entries in the form (e, key) , where e is an index node and key is defined as the negative score upper bound, $e.UB$, of node e (line 2). To retrieve PTD candidates, we first insert the root, $root(I_i)$, of the aR-tree index I_i into heap H , in the form of $(root(I_i), 0)$ (line 3), and then utilize heap H to traverse the index I_i (lines 4–30). Each time we pop out an entry (e, key) with the minimal key value from heap H (line 5). If the score upper bound, $e.UB$, of the entry e is less than or equal to threshold τ_i , then we will terminate the loop (since all nodes in heap H have scores lower than at least k candidates; lines 6–8).

When node e is a leaf node, for each uncertain object $t \in e$, we can compute its score lower/upper bounds, $t.LB$ and $t.UB$, and maintain a set, S_{cand} , of at least k PTD candidates (lines 9–25). That is, we set τ_i to the k -th largest score lower bound in S_{cand} (line 17 or 23), and use τ_i to filter out those objects t with score upper bound not greater than τ_i , by applying the score bound pruning method (lines 20–25).

Similarly, when we encounter a non-leaf node e , we check each child entry $e_r \in e$ (lines 27–30). If it holds that $e_r.UB \leq \tau_i$, then we can safely prune node e_r ; otherwise, we insert entry $(e_r, -e_r.UB)$ into heap H for further checking (lines 29–30).

After the index traversal, we consider each candidate t in the candidate set S_{cand} and obtain a list of nodes eid that are partially dominated by instances $t_j \in t$ (lines 31–33). If nodes eid are from partition D_l , we will set the output key of this algorithm to l , and value to $\{t, t_j, eid, t.LB, t.UB, \tau_i\}$. We will emit the resulting key-value pairs to partition D_l for further computation of scores of $S(t)$ (lines 34–35).

Complexity analysis The filtering-mapper function in Algorithm 2 traverses the index I_i of partition D_i . Assuming that partition D_i contains M_i uncertain objects, the tree index I_i has about $O(\frac{M_i}{F_i})$ nodes, where F_i is the average fanout in the R*-tree I_i . Thus, in the worst case, the time complexity of traversing the index (lines 4–30) is given by $O(M_i)$ (by considering F entries per node on average). Moreover, the time complexity of lines 31–35 is given by $O(|S_{cand}| \cdot \sum_{l=1}^N \frac{M_l}{F_l})$, where $|S_{cand}|$ is the size of candidate set S_{cand} . Therefore, the total time complexity of filtering-mapper function is $O(M_i + |S_{cand}| \cdot \sum_{l=1}^N \frac{M_l}{F_l})$.

Algorithm 2: Filtering-Mapper Function on Partition D_i

Input: Key: partition number i , **Value:** uncertain database D_i , indexes I_l of partitions D_l ($1 \leq l \leq N$), a query point q , and integer k

Output: key-value pairs with PTD candidate instances and their partially dominated MBRs

```

1  $S_{cand} = \emptyset$ ,  $cnt = 0$ ,  $\tau_i = -\infty$ ;
2 initialize an empty min-heap  $H$  with entry form  $(e, key)$ 
3 insert  $(root(I_i), 0)$  into heap  $H$ 
4 while  $H$  is not empty do
5    $(e, key) = \text{de-heap}(H)$ 
6    $e.UB = -key$ ;
7   if  $e.UB \leq \tau_i$  then
8      $\perp$  terminate the loop;
9   if  $e$  is a leaf node then
10    for each uncertain object  $t \in e$  do
11      if  $cnt < k$  then
12         $t.LB = \text{ScoreLB}(q, t, I_1, \dots, I_N)$  // Eq.(3)
13         $t.UB = \text{ScoreUB}(q, t, I_1, \dots, I_N)$  // Eq.(4)
14        add  $(t, t.LB, t.UB)$  to  $S_{cand}$ ;
15         $cnt++$ ;
16        if  $cnt == k$  then
17           $\perp$  let  $\tau_i$  be the  $k$ -th largest score lower bound in  $S_{cand}$ 
18      else
19         $t.UB = \text{ScoreUB}(q, t, I_1, \dots, I_N)$  //Eq.(4)
20        // score bound pruning
21        if  $t.UB > \tau_i$  then
22           $t.LB = \text{ScoreLB}(q, t, I_1, \dots, I_N)$  //Eq.(3)
23          add  $(t, t.LB, t.UB)$  to  $S_{cand}$ ;
24          update  $\tau_i$  with the  $k$ -th largest score lower bound in  $S_{cand}$ 
25          use  $\tau_i$  to prune candidates  $t' \in S_{cand}$  with  $t'.UB < \tau_i$ 
26           $cnt = |S_{cand}|$ ;
27    else
28      //  $e$  is a non-leaf node
29      for each child node  $e_r \in e$  do
30         $e_r.UB = \text{ScoreUB}(q, e_r, I_1, \dots, I_N)$ ; // score bound pruning
31        if  $e_r.UB > \tau_i$  then
32           $\perp$  insert  $(e_r, -e_r.UB)$  into heap  $H$ 
33 for each uncertain object  $t \in S_{cand}$  do
34   for each instance  $t_j \in t$  do
35     obtain a list,  $t_j.list$ , of node/object IDs,  $eid$ , from indexes  $I_l$  that are partially dominated by  $t_j$ ,
      in the form  $(l, eid)$ 
36     for each entry  $(l, eid) \in t_j.list$  do
37        $\perp$  emit the key-value pair  $(l, \{t, t_j, eid, t.LB, t.UB, \tau_i\})$ 

```

The communication cost depends on the output of Algorithm 2, which is given by $O(|S_{cand}| \cdot |t| \cdot |t_j.list|)$, where $|t|$ is the average number of instances per uncertain object, and $|t_j.list|$ is the average number of nodes/objects partially dominated by an instance t_j .

Algorithm 3: Filtering-Reducer Function

Input: Key: partition ID D_l , **Value:** $\{t, t_i, eid, t.LB, t.UB, \tau_i\}$
Output: Key: object t **Value:** $\{t.LB, t.\Delta S, \tau_i\}$

```

1 group all the instances  $t_i$  based on their objectID  $t$ 
2  $t.\Delta S = 0$ 
3  $\tau = \max_{l=1}^N \tau_l$ 
4 for each instance  $t_i \in t$  do
5    $t_i.pScore = 0$ ;
6   load node  $eid$  from index  $I_l$ 
7   // initialize heap  $H$  with child nodes  $e$  of  $eid$ 
8   for each child  $e \in eid$  do
9     insert  $(e, e.level)$  into heap  $H$ 
10  while  $H$  is not empty do
11     $(e, key) = \text{de-heap}(H)$ 
12    if  $e$  is an uncertain object  $o$  then
13      for each instance  $o_j \in o$  do
14        if  $t_i$  dynamically dominates  $o_j$  then
15           $t_i.pScore = t_i.pScore + t_i.p \cdot o_j.p$ ;
16    else
17      for each child  $e_j \in e$  do
18        if  $t_i$  fully dominates  $e_j$  w.r.t.  $q$  then
19           $t_i.pScore = t_i.pScore + t_i.p \cdot e_j.sum$ ;
20        else
21          if  $t_i$  partially dominates  $e_j$  w.r.t.  $q$  then
22            insert  $(e_j, e_j.level)$  into heap  $H$ 
23    // update the score upper bound of  $t$ 
24     $t.UB = t.UB - eid.sum \cdot t_i.p + t_i.pScore$ 
25    if  $t.UB < \tau$  then
26      emit  $(t, \emptyset)$ 
27      return;
28   $t.\Delta S = t.\Delta S + t_i.pScore$ 
29 emit the key-value pair  $(t, \{t.LB, t.\Delta S, \tau\})$ 

```

6.2 Filtering-reducer function

Algorithm 3 shows the pseudo code of the reducer function: filtering-reducer (running on each of N servers), which aims to compute the partial scores with respect to the received candidate instances t_i and their partially dominated MBRs eid . It takes partition ID D_l as the key and (object ID t , instance ID t_i , node ID eid , object score lower bound $t.LB$, object score upper bound $t.UB$) as the value. The output is key-value pairs (object ID t , {object score lower bound $t.LB$, partial score $t.\Delta S$, threshold τ }).

In particular, the mapper algorithm first obtains all instances t_i of the same PTD candidates t from the filtering-mapper phase (line 1). We use $t.\Delta S$ to record partial scores w.r.t. instances t_i and their partially dominated MBRs eid , which is initially 0 (line 2). Moreover, we let τ be the maximum value among all thresholds τ_l from input key-value pairs (i.e., $\max_{l=1}^N \tau_l$), which can be used to enable the score bound pruning (line 3).

Next, we calculate the partial score $t.\Delta S$ for all the received instances $t_i \in t$ (lines 4–26). Initially, we load the partially dominated MBR node eid from index I_l (line 6), and insert children e of node eid into an initially empty min-heap H in the form $(e, e.level)$ (lines 7–8),

Algorithm 4: Refinement-Mapper Function

Input: Key: t **Value:** a list, L , of set values $\{t.LB, t.\Delta S, \tau\}$
Output: Key: D_1 **Value:** $\{t, S(t)\}$

```

1 if list  $L$  contains an empty set  $\emptyset$  then
2    $\mid$  return;
3  $S(t) = t.LB$ ;
4 for each set  $\{t.LB, t.\Delta S, \tau\} \in L$  do
5    $\mid$   $S(t) = S(t) + t.\Delta S$ 
   // score bound pruning
6 if  $S(t) < \tau$  then
7    $\mid$  return;
8 emit( $D_1, \{t, S(t)\}$ )

```

where $e.level$ is the level of the node e (here, $e.level = 0$, when e is a leaf node; $e.level > 0$, when e is a non-leaf node). Intuitively, we set the key of the heap entry to $e.level$, since we want to quickly descend to leaf nodes during the index traversal.

Each time we pop out an entry (e, key) from the heap H (lines 9–10). If e is an uncertain object o , then we can calculate the partial score that t_i dominates object o , which is given by summing up the existence probabilities of instances $o_j \in o$ dominated by t_i (lines 11–14). On the other hand, if e is a leaf or non-leaf node, then we check each child $e_j \in e$ (lines 16–21). If t_i fully dominates e_j (i.e., node e_j is in $DDR(t_i)$), then we include aggregate $t_i.p \cdot e_j.sum$ in the partial score $t_i.pScore$ (lines 17–18); otherwise (i.e., t_i partially dominates e_j), we insert $(e_j, e_j.level)$ into heap H for further checking (lines 19–21).

After computing the partial score w.r.t. t_i and eid , we can now update and obtain tighter score upper bound, $t.UB$, of object t (line 22). By applying the score bound pruning, if t can be safely pruned (i.e., $t.UB < \tau$), then we emit $(t, \emptyset)$, which indicates that object t can be ruled out (lines 23–25).

Finally, for all instances t_i , we can obtain the total partial score $t.\Delta S$ (line 26) and emit a key-value pair in the form $(t, \{t.LB, t.\Delta S, \tau\})$ for combining partial scores of object t in the next refinement round (i.e., refinement-mapper; line 27).

Complexity analysis The time complexity of filtering-reducer function in Algorithm 3 is given by $O(|t| \cdot F_l^{eid.level})$, where $|t|$ is the number of instances in object t , F_l is the average fanout of index I_l , and $eid.level$ is the level of node eid in index I_l .

The communication cost resulting from the filtering-reducer function is given by $O(1)$ (due to the emitted key-value pair in line 27).

6.3 Refinement-mapper function

Algorithm 4 illustrates the pseudo-code of a map-only job, refinement-mapper function, in a second round, which combines the partial scores, $t.\Delta S$, of instances and their partially dominated MBRs into an exact score $S(t)$. Specifically, the input of this algorithm takes object ID t as the key, and a list, L , of set values $(t.LB, t.\Delta S, \tau)$ as the value. Moreover, the output of this algorithm returns object t and its exact score $S(t)$ to the master node.

In particular, for the received key-value pairs, if the list L contains an empty set $\emptyset$ (which was emit by line 24 of Algorithm 3), it indicates that uncertain object t can be safely pruned and thus ignored (lines 1–2). Otherwise, we first set $S(t)$ to the score lower bound $t.LB$ (line 3), and then add to $S(t)$ the partial scores $t.\Delta S$ for sets $\{t.LB, t.\Delta S, \tau\} \in L$ (lines 4–5). If

$S(t) < \tau$ holds, then we can apply the score bound pruning method (given in Lemma 2) to filter out uncertain object t ; otherwise, we can emit object t and its exact score $S(t)$ to the master server with D_1 (lines 6–8).

Complexity analysis The time complexity of the refinement-mapper function in Algorithm 4 is given by $O(|t|)$, where $|t|$ is the number of instances in uncertain object t . Moreover, the communication cost (for line 8) is given by $O(1)$.

7 Experimental evaluation

In this section, we test our proposed distributed *PTD* query framework over both real and synthetic data sets.

Real/synthetic data sets We used real and synthetic datasets for our experimental evaluation. For real data, we tested California road networks [8], denoted as *Real*, which contains 98,451 MBRs for road segments in California. These MBRs can be considered as uncertainty regions of uncertain objects. Then, we randomly generate instances (samples) inside MBRs, each instance associated with equal probability, where the summed appearance probability of all instances in each MBR is equal to 1.

For synthetic data, we generate uncertain objects t as follows. In particular, for each uncertain object t , we generate the center C_t of its uncertainty region $UR(t)$ (bounding instances), following Uniform, Gaussian, or Zipf (with skewness 0.8) distribution. Then, we also randomly produce the side length of $UR(t)$ within $(0, l_{max}]$. This way, we can obtain uncertainty regions $UR(t)$ of uncertain objects t , and uniformly generate instances within $UR(t)$, where the number of instances per object is within $[|t|_{min}, |t|_{max}]$. With different data distributions, we can obtain 3 types of data sets, denoted as *uniform*, *gaussian*, and *zipf*, respectively. For each real/synthetic data set, we randomly distribute uncertain objects to N servers and construct indexes over N partitions.

Measures To evaluate the *PTD* query performance, we will randomly generate 20 query points in the data space. We report the *wall clock time* and *communication cost*. Here, the wall clock time is the time cost of answering the *PTD* query by taking the average over 20 queries, whereas the communication cost is the average size of communication data among servers.

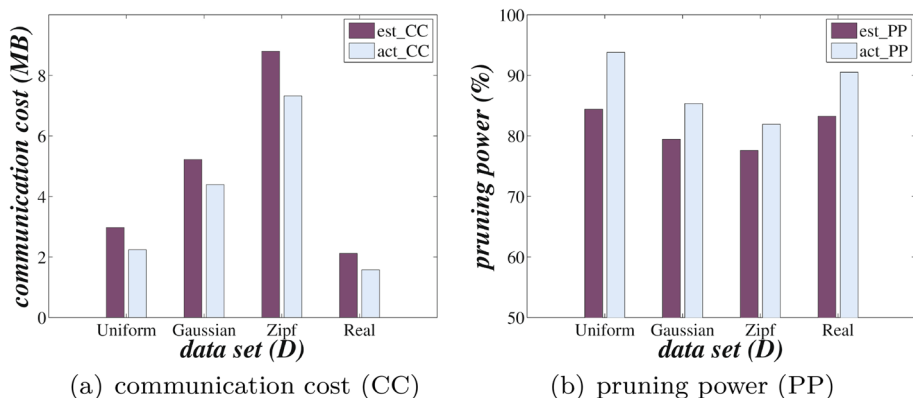
Competitor We will compare our distributed *PTD* algorithm with several baseline algorithms, first is the *baseline*, which is a naive algorithm (mentioned in Sect. 1), which retrieves all data and indexes from other $(N - 1)$ servers, and conducts *PTD* queries on a single server. We also compare with two other baseline algorithms, *PTD_WoP* and *PTD_WoI*, which do not use pruning methods and indexing, respectively.

Network settings We assume that the network conditions have high bandwidth, and do not consider abnormal network scenarios (e.g., with network congestion).

Parameter settings Table 2 depicts the parameter settings, where the default values are in bold. Each time we vary the values of one parameter, while other parameters are set to their default values. We ran experiments on 10 dell PowerEdge R730 servers with Intel Xeon

Table 2 The parameter settings

Parameters	Values
N	1, 2, 5, 8, 10
l_{max}	1, 2, 3 , 4, 5
k	5, 10, 15 , 20, 25
$[t _{min}, t _{max}]$	[2,5], [2,8], [2,10], [2,12], [2,15]
$ D $	100K, 200K, 500K , 800K, 1 M

**Fig. 6** The cost model verification for the communication cost (CC) and pruning power (PP) versus real/synthetic data sets

E5-2630 2.2GHz CPU and 64 GB memory. All algorithms were compiled by Java 1.8 and used Hadoop 1.2.1 for MapReduce.

7.1 The cost model verification

In the first set of experiments, we verify the effectiveness of our proposed cost model for the index distribution (as discussed in Sect. 5.2). Specifically, Fig. 6a compares the estimated communication cost, denoted as est_CC (given by Eq. (7)) with the actual one, act_CC ; Fig. 6b shows the comparison of estimated pruning power, est_PP , with the actual one, act_PP . Here, we obtain the estimated pruning power given by $est_PP = \frac{|D_I| - |S_{cand}|}{|D_I|}$ for each partition D_I , where $|S_{cand}|$ is estimated by Eq. (9). From figures, we can see that both estimated communication costs and pruning powers over real/synthetic data sets can closely mimic the actual ones (and their trends), which confirms the correctness and effectiveness of our cost model (i.e., Eqs. (7) and (9)) for guiding the selection of levels in indexes.

7.2 The evaluation of the efficiency

The PTD performance versus real/synthetic data sets Figure 7 shows the performance of our distributed PTD algorithm and baseline algorithms, namely $baseline$, PTD_WoP , and PTD_WoI , over real and synthetic data sets, where parameters are set to default values for synthetic data. From figures, we can see that our PTD algorithm outperforms all the baseline algorithms by about an order of magnitude, in terms of both wall clock time and

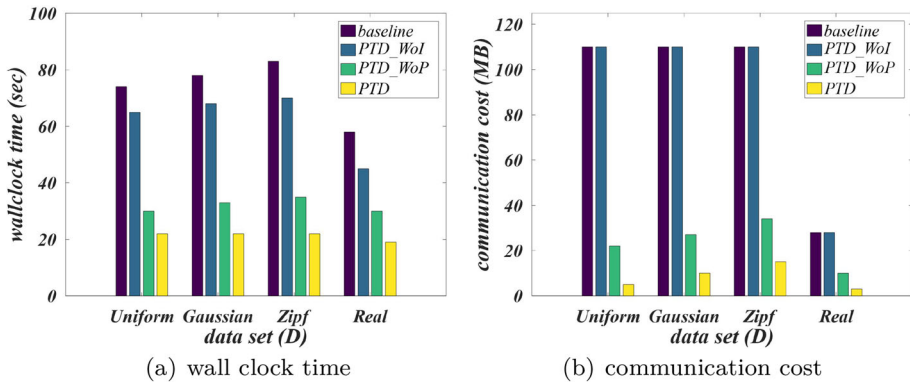


Fig. 7 The PT D performance versus real/synthetic data sets

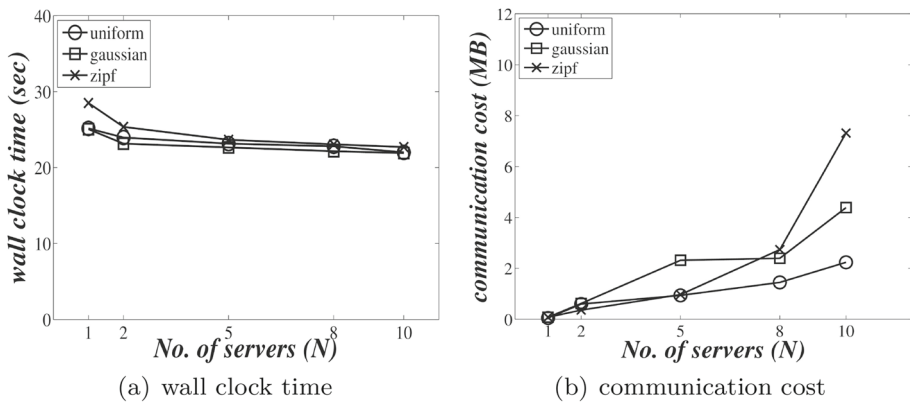


Fig. 8 The PT D performance versus no. of servers N

communication cost. This confirms the effectiveness of our proposed technique such as pruning methods and indexing mechanisms, as well as the efficiency of our distributed PT D approach over both real and synthetic data sets.

Below, to clearly show the trends and robustness of our PT D approach with respect to different parameters, we will only report PT D experimental results over synthetic data sets, by testing different parameters (such as N , l_{max} , k , $[|t|_{min}$, $|t|_{max}]$, and $|D|$).

The PT D performance versus no. of servers N . Figure 8 evaluates the wall clock time and communication cost of our PT D algorithm over *uniform*, *gaussian*, and *zipf* data sets, by varying the number of servers (i.e., N) from 1 to 10, where default values are used for other parameters (as depicted in Table 2). From the figure, we can see that running the PT D query on a single server (i.e., $N = 1$) is more costly. This is because the processing on one server cannot take advantage of parallel computation over multiple servers. With the increase of N , the amount of partition data in D_i to be processed for each server decreases, and multiple servers can answer the PT D query in parallel. Therefore, the wall clock time becomes smaller for larger N value. On the other hand, since each server returns at least k PT D candidates, for more servers (i.e., larger N), the total number of PT D candidates tends to increase, which leads to higher communication cost.

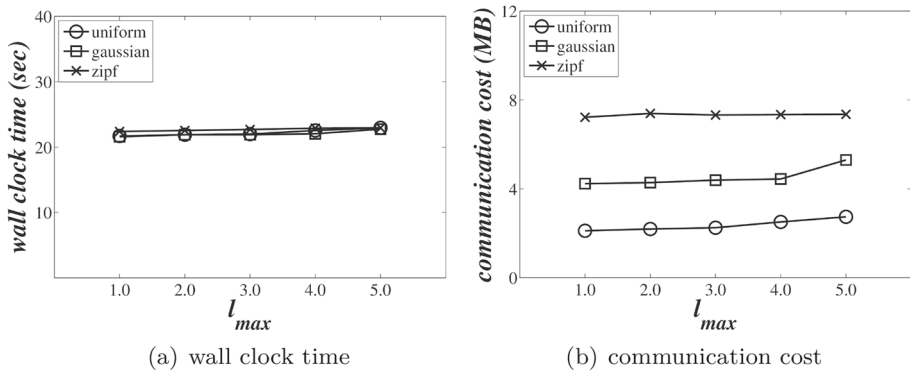


Fig. 9 The PTD performance versus l_{max}

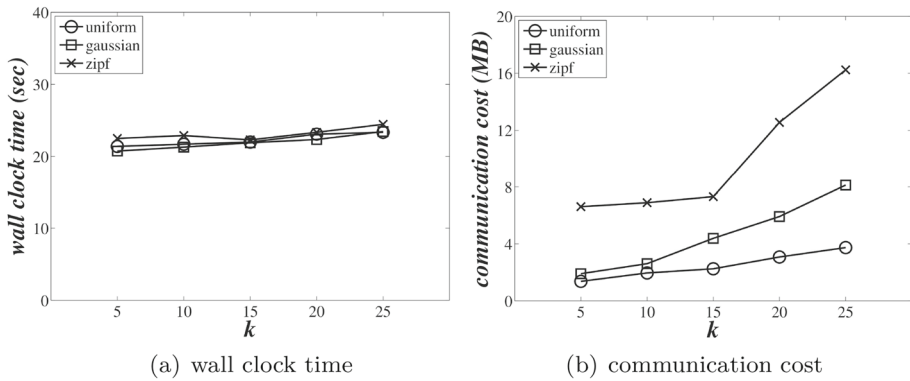


Fig. 10 The PTD performance versus parameter k

The PTD performance versus maximum side length, l_{max} , of uncertain objects Figure 9 varies the maximum possible side length, l_{max} , of uncertainty regions from 1 to 5, where other parameters are set to default values. With larger uncertainty regions, uncertain objects have higher chance of overlapping. Thus, in figures, with the increase of the l_{max} value, both wall clock time and communication cost tend to slightly increase.

The PTD performance versus k . Figure 10 illustrates the effect of parameter k in top- k dominating queries on the performance of our *PTD* algorithm, where $k = 5, 10, 15, 20$, and 25 and other parameters are set to default values. When we increase k , the wall clock time also smooth increases, whereas the communication cost increases slightly for *uniform* data set, but sharply for *Gaussian* and *Zipf* data distributions (due to the skewed data distributions).

The PTD performance versus the range, $[|t|_{min}, |t|_{max}]$, for no. of instances per object Figure 11 shows the wall clock time and communication cost of our *PTD* algorithm over synthetic data sets, by varying the range, $[|t|_{min}, |t|_{max}]$, for the number $|t|$ of instances per uncertain object t from $[2, 5]$ to $[2, 15]$, where default values are used for other parameters (as depicted in Table 2). The experimental results show that, with more instances per object, the execution time and communication cost are increasing. This is reasonable, since more

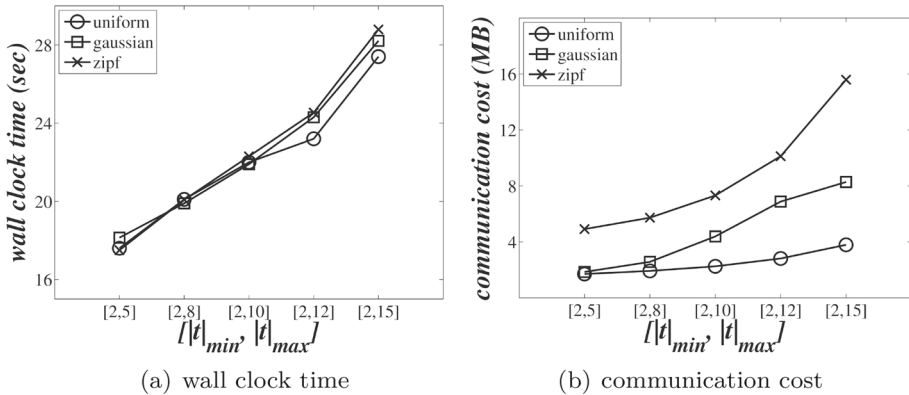


Fig. 11 The PTD performance versus the range, $[t_{min}, t_{max}]$, for No. of instances per uncertain object

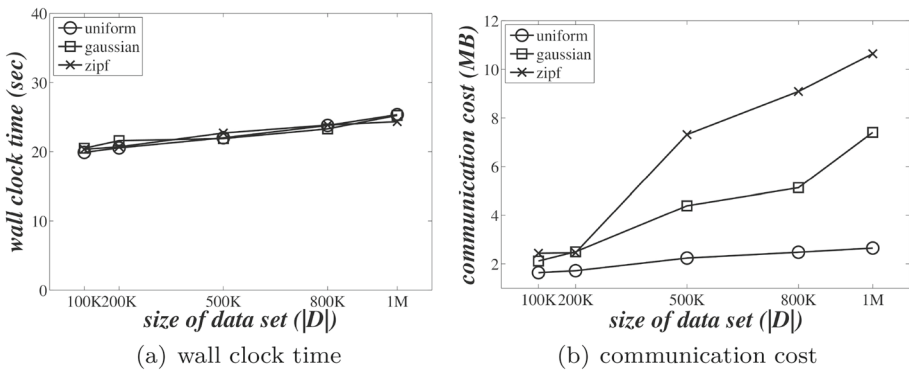


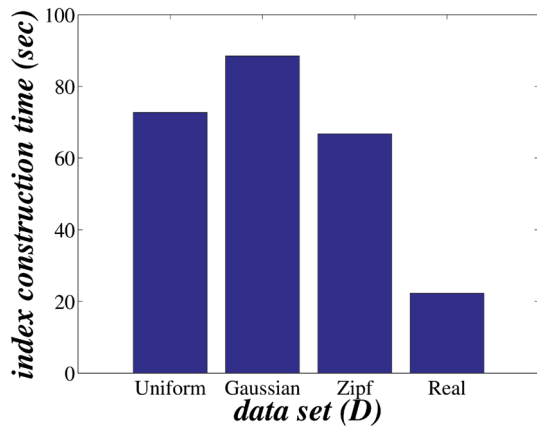
Fig. 12 The PTD performance versus the size, $|D|$, of data set

instances in uncertain objects should be processed and more key-value pairs will be emitted among servers during the distributed PTD query processing.

The PTD performance versus the number, $|D|$, of uncertain objects Figure 12 evaluates the scalability of our PTD algorithm by varying the number, $|D|$, of uncertain objects from 100K to 1M, where other parameters are set to their default values. Note that, each uncertain object has about 2–10 instances, which indicates that the total number of instances can be up to $2M - 10M$ in uncertain database D . From figures, when the data size $|D|$ becomes larger, the wall clock time of our PTD algorithm slightly increases (from 20 sec to 25 sec), whereas the communication cost increases from about 2 MB to 10 MB. This is reasonable, since larger data sets lead to more candidates and thus higher time/communication costs to filter and refine.

The index construction time versus real/synthetic data set D . Finally, we also report the index construction time in Fig. 13, for different real and synthetic data sets, where parameters of synthetic data are set to default values. Note that, the index construction time of the real data set is lower than that of synthetic data, due to smaller size (i.e., $|D| = 98,451$) of real data set (compared with $|D| = 500K$ for synthetic data). For all real/synthetic data sets, the offline index construction times are around 22 ~ 88 s.

Fig. 13 The index construction time versus real/synthetic data sets



To summarize, our *PTD* algorithm is scalable and robust against different parameters. We also tested other parameters (e.g., other data distributions) with similar experimental results, which will not be reported here.

8 Related work

In this section, we overview previous works on the top- k dominating query in centralized certain/uncertain databases, and query processing in centralized and distributed uncertain databases.

8.1 Top- k dominating query on centralized certain/uncertain data

Top- k dominating query on certain data In the centralized environment, Papadias et al. [30] first proposed a variant of the skyline query, that is, the top- k dominating query, over certain data. The top- k dominating query takes the advantages of both skyline and top- k queries, which is not sensitive to scales of dimensions, does not require users to specify a ranking function, and can control the size k of the returned query answers. Yiu and Mamoulis [43] considered the top- k dominating query over certain data, where attributes of objects are static coordinates (which is different from our work that takes into account dynamic attributes of objects w.r.t. query point q). In particular, they utilized aggregates in the aR-tree index to estimate the lower/upper bounds of objects' scores, which can be used for pruning and facilitate the top- k dominating query answering.

In addition, there are some other variants of top- k dominating queries over certain data, such as the top- k dominating query assuming that objects are vertically decomposed [16, 39], that in the metric space [40], continuous top- k dominating query [36], and so on.

Top- k dominating query on uncertain data In the literature of uncertain databases, there are some related works [12, 27, 28, 45] on probabilistic top- k dominating queries (PTD) under *possible worlds* [10] semantics. Specifically, Lian and Chen [27, 28] studied the PTD query (and its variant in arbitrary subspaces) over uncertain objects with dynamic attributes with respect to a query point q . Zhang et al. [45] considered a variant of the PTD query, *threshold-based probabilistic top- k dominating query* (PtopkQ), over uncertain objects with static attributes (i.e., coordinates), which retrieves k objects o with the highest scores, where the score is given by the maximum number of objects dominated by uncertain object o with

probability greater than a user-specified threshold. Moreover, Feng et al. [12] studied the PTD query over sliding windows of uncertain data streams.

All prior works mentioned above designed PTD pruning heuristics, and investigated efficient processing of PTD queries via indexes on a single machine. However, in practice, uncertain databases are of large scale in many real-world applications, and/or sometimes uncertain data are collected and stored in distributed servers. Therefore, PTD query processing on centralized uncertain database may not meet the strict requirement of low response time in real applications. Our work in this paper exactly formulates and tackles the PTD problem over uncertain databases in a distributed environment, which cannot directly borrow previous PTD techniques on certain data or centralized uncertain data.

8.2 Query processing on centralized and distributed uncertain databases

Centralized Uncertain Data Management. In the literature, many systems such as MystiQ [4], Orion [7], TRIO [3], MayBMS [1], MCDB [18], and BayesStore [41] have been proposed to manipulate probabilistic/uncertain data. The data uncertainty can be classified into two categories, [37], *attribute uncertainty* and *tuple uncertainty*. The attribute uncertainty usually involves (spatial) uncertain databases, whereas the tuple uncertainty is often modeled by probabilistic (relational) databases. In uncertain databases, each object is represented by a spatial *uncertainty region*, in which the uncertain object resides following some probabilistic distribution. This object distribution can be captured by either discrete samples (associated with existence probabilities) [17, 26, 35] or a continuous *probability density function* (pdf) [5, 11]. In this paper, we consider the tuple uncertainty, that is, probabilistic databases [10], which consist of x -tuples, each containing mutually exclusive tuples (or *alternatives*) with existence probabilities.

Similarly, [13, 15, 31, 32] tried to manipulate inconsistent probabilistic spatiotemporal databases. Specifically, [13] studied the issue of incorporating denial constraints into probabilistic databases. [15] studied the count queries on probabilistic spatiotemporal knowledge bases. [31] focused on knowledge representation in the form of integrity constraints for probabilistic spatiotemporal data. [32] focused on computing consistent answers and repairs based on probability interval expansion over inconsistent probabilistic spatiotemporal databases. Since these papers focused on knowledge representation over probabilistic spatiotemporal databases and studied different queries other than the PTD query, we cannot directly apply them to our PTD query answering.

Query processing in uncertain databases has been extensively studied in many real applications. In particular, various probabilistic query types have been explored over uncertain data, including *probabilistic range query* [5, 6], *probabilistic nearest neighbor query* [5, 6, 20], *probabilistic top-k query* [9, 17, 25], *probabilistic skyline query* [35], *consistent query answering* [29], and so on. In these works, specific for different query types, effective pruning methods are often designed to reduce the problem search space, and uncertain objects are organized by multidimensional indexes to facilitate efficient query processing algorithms on a centralized machine. Also, some queries such as top- k or k -nearest neighbor queries require defining distance/ranking functions which are not trivial how to the balance between different attributes.

Previous techniques on a centralized machine cannot be directly used in our PTD query over distributed servers, since we need to design novel and specific MapReduce functions for the distributed PTD query answering.

Distributed uncertain data management In the distributed environment, prior works [33, 34, 44, 46] studied how to answer skyline and/or reverse skyline queries in parallel over probabilistic data, following the MapReduce framework. Similarly, Li et al. [24] tackled the distributed top- k queries over uncertain data. Lai et al. [21] studied the PTD problem over multiple uncertain IoT Data Streams in Edge Computing Environment. Although [21] tackles the *PTD* query, it considered the scenario of uncertain IoT data streams whose size is much smaller than that of distributed uncertain database in our work. The work in [21] also required to monitor the *PTD* query continuously in real time and used edge computing, which are different from our MapReduce framework. To the best of our knowledge, no previous works tackled the PTD problem over distributed uncertain databases. Due to different problem definitions, we cannot directly apply previous techniques on other queries or *PTD* in a different scenario to solving our *PTD* problem in a distributed setting.

9 Conclusion

In this paper, we study the problem of probabilistic top- k dominating (PTD) query over large-scale and distributed uncertain database. In order to efficiently and effectively tackle the PTD problem, we design a cost model for the index distribution with the purposes of maximizing the pruning power and minimizing the communication cost during the PTD query processing. We also propose an effective MapReduce framework for answering distributed PTD queries, which consists of specifically designed mapper and reducer functions that incorporates pruning strategies to reduce the PTD search space. Extensive experiments have been conducted to show the efficiency and effectiveness of our proposed PTD approaches over both real and synthetic data sets.

Acknowledgements Funding for this work was provided by NSF OAC No. 1739491, NSF CCF No. 2217104, and Lian Startup No. 220981, Kent State University.

References

1. Antova L, Koch C, Olteanu D (2007) MayBMS: Managing incomplete information with probabilistic world-set decompositions. In: IEEE international conference on data engineering
2. Arenas M, Bertossi LE, Chomicki J (1999) Consistent query answers in inconsistent databases. In: Proceedings of the eighteenth ACM SIGACT-SIGMOD-SIGART symposium on principles of database systems (PODS)
3. Benjelloun O, Sarma AD, Halevy A, Widom J (2006) ULDBs: databases with uncertainty and lineage. In: Proceedings of the very large data bases
4. Boullos J, Dalvi N, Mandhani B, Mathur S, Ré C, Suciu D (2005) Mystiq: a system for finding more answers by using probabilities. In: Proceedings of ACM SIGMOD international conference on management of data
5. Cheng R, Kalashnikov DV, Prabhakar S (2003) Evaluating probabilistic queries over imprecise data. In: Proceedings of ACM SIGMOD international conference on management of data
6. Cheng R, Kalashnikov DV, Prabhakar S (2004) Querying imprecise data in moving object environments. IEEE Transactions on Knowledge and Data Engineering **16**(9)
7. Cheng R, Singh S, Prabhakar S (2005) U-DBMS: a database system for managing constantly-evolving data. In: Proceedings of the very large data bases
8. Chorochronos.org: Tiger/line: California streets mbr data (2018). <http://chorochronos.datastories.org/?q=node/59>. Accessed 15 Apr 2018
9. Cormode G, Li F, Yi K (2009) Semantics of ranking queries for probabilistic data and expected ranks. In: IEEE international conference on data engineering
10. Dalvi N, Suciu D (2007) Efficient query evaluation on probabilistic databases. VLDB J. **16**(4)
11. Deshpande A, Guestrin C, Madden SR, Hellerstein JM, Hong W (2004) Model-driven data acquisition in sensor networks. In: Proceedings of the very large data bases

12. Feng X, Zhao X, Gao Y, Zhang Y (2013) Probabilistic top-k dominating query over sliding windows. In: Proceedings of web technologies and applications—15th Asia-Pacific Web Conference (APWeb)
13. Flesca S, Furfaro F, Parisi F (2014) Consistency checking and querying in probabilistic databases under integrity constraints. *J Comput Syst Sci* 80(7):1448–1489. <https://doi.org/10.1016/j.jcss.2014.04.026>
14. Fuxman A, Fazli E, Miller RJ (2005) Conquer: Efficient management of inconsistent databases. In: Proceedings of the ACM SIGMOD international conference on management of data
15. Grant J, Molinaro C, Parisi F (2018) Probabilistic spatio-temporal knowledge bases: capacity constraints, count queries, and consistency checking. *Int J Approx Reason* 100:1–28. <https://doi.org/10.1016/j.ijar.2018.05.003>
16. Han X, Li J, Gao H (2015) TDEP: efficiently processing top-k dominating query on massive data. *Knowl Inf Syst* 43(3)
17. Hua M, Pei J, Zhang W, Lin X (2008) Ranking queries on uncertain data: a probabilistic threshold approach. In: Proceedings of ACM SIGMOD international conference on management of data
18. Jampani R, Xu F, Wu M, Perez LL, Jermaine C, Haas PJ (2008) MCDB: a Monte Carlo approach to managing uncertain data. In: Proceedings of ACM SIGMOD international conference on management of data
19. Karp RM, Shenker S, Papadimitriou CH (2003) A simple algorithm for finding frequent elements in streams and bags. *ACM Trans Database Syst* 28(1)
20. Kriegel HP, Kunath P, Renz M (2007) Probabilistic nearest-neighbor query on uncertain objects. In: Database systems for advanced applications (DASFAA)
21. Lai CC, Wang TC, Liu CM, Wang LC (2019) Probabilistic top-k dominating query monitoring over multiple uncertain IoT data streams in edge computing environments. *IEEE Internet Things J* 6(5):8563–8576. <https://doi.org/10.1109/jiot.2019.2920908>
22. Lazaridis I, Mehrotra S (2001) Progressive approximate aggregate queries with a multi-resolution tree structure. In: Proceedings of ACM SIGMOD international conference on management of data
23. LeFevre K, DeWitt DJ, Ramakrishnan R (2005) Incognito: Efficient full-domain k-anonymity. In: Proceedings of the ACM SIGMOD international conference on management of data
24. Li F, Yi K, Jestes J (2009) Ranking distributed probabilistic data. In: Proceedings of ACM SIGMOD international conference on management of data
25. Li J, Saha B, Deshpande A (2011) A unified approach to ranking in probabilistic databases. *VLDB J*. 20(2)
26. Lian X, Chen L (2008) Monochromatic and bichromatic reverse skyline search over uncertain databases. In: Proceedings of ACM SIGMOD international conference on management of data
27. Lian X, Chen L (2009) Top-k dominating queries in uncertain databases. In: International conference on extending database technology (EDBT)
28. Lian X, Chen L (2013) Probabilistic top-k dominating queries in uncertain databases. *Inf Sci* 226
29. Lian X, Chen L, Song S (2010) Consistent query answers in inconsistent probabilistic databases. In: Proceedings of the 2010 ACM SIGMOD international conference on management of data, SIGMOD'10, pp 303–314. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/1807167.1807202>
30. Papadias D, Tao Y, Fu G, Seeger B (2005) Progressive skyline computation in database systems. *ACM Trans Database Syst* 30(1)
31. Parisi F, Grant J (2016) Knowledge representation in probabilistic spatio-temporal knowledge bases. *J Artif Intell Res* 55:4883. <https://doi.org/10.1613/jair.4883>
32. Parisi F, Grant J (2017) On repairing and querying inconsistent probabilistic spatio-temporal databases. *Int J Approx Reason* 84:41–74. <https://doi.org/10.1016/j.ijar.2017.02.003>
33. Park Y, Min JK, Shim K (2013) Parallel computation of skyline and reverse skyline queries using mapreduce. In: Proceedings of the very large data bases, vol 6(14)
34. Park Y, Min JK, Shim K (2015) Processing of probabilistic skyline queries using mapreduce. In: Proceedings of the very large data bases, vol 8(12)
35. Pei J, Jiang B, Lin X, Yuan Y (2007) Probabilistic skylines on uncertain data. In: Proceedings of the very large data bases
36. Santoso BJ, Chiu G (2014) Close dominance graph: an efficient framework for answering continuous top-k dominating queries. *IEEE Trans Knowl Data Eng* 26(8)
37. Singh S, Mayfield C, Shah R, Prabhakar S, Hambrusch S, Neville J, Cheng R (2008) Database support for probabilistic attributes and tuples. In: IEEE international conference on data engineering
38. Skoutas D, Sacharidis D, Simitis A, Kantere V, Sellis T (2009) Top-k dominant web services under multi-criteria matching. In: International conference on extending database technology (EDBT)
39. Tiakas E, Papadopoulos AN, Manolopoulos Y (2011) Progressive processing of subspace dominating queries. *VLDB J*. 20(6)

40. Tiakas E, Valkanas G, Papadopoulos AN, Manolopoulos Y (2014) Metric-based top- k dominating queries. In: Proceedings of the 17th international conference on extending database technology (EDBT)
41. Wang DZ, Michelakis E, Garofalakis M, Hellerstein J (2008) Bayestore: managing large, uncertain data repositories with probabilistic graphical models. In: Proceedings of the very large data bases
42. Wikipedia: Central limit theorem—wikipedia, the free encyclopedia (2017). https://en.wikipedia.org/w/index.php?title=Central_limit_theorem&oldid=800332726. Accessed 15 Sep 2017
43. Yiu ML, Mamoulis N (2007) Efficient processing of top- k dominating queries on multi-dimensional data. In: Proceedings of the very large data bases
44. Zhang J, Jiang X, Ku WS, Qin X (2016) Efficient parallel skyline evaluation using mapreduce. *IEEE Trans Parallel Distrib Syst* 27(7)
45. Zhang W, Lin X, Zhang Y, Pei J, Wang W (2010) Threshold-based probabilistic top- k dominating queries. *VLDB J* 19(2)
46. Zhou X, Li K, Zhou Y, Li K (2016) Adaptive processing for distributed skyline queries over uncertain data. *IEEE Trans Knowl Data Eng* 28(2)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Niranjan Rai received the BE degree in Computer Engineering from the Institute of Engineering, Tribhuvan University, Nepal, and the PhD degree in Computer Science from Kent State University, Ohio, USA.



Xiang Lian received the BS degree from the Department of Computer Science and Technology, Nanjing University, China, and the PhD degree in Computer Science from the Hong Kong University of Science and Technology, Hong Kong. He is now an associate professor in the Computer Science Department at Kent State University, USA. His research interests include probabilistic data management, spatio-temporal databases, uncertain/certain graph databases, and so on.